

Real-Time Data Sanity Checks

A sanity check is an automated test that ensures live data is valid, complete, and correctly formatted before downstream systems use it.

1. Speed & Latency

Problem: Slow database lookups during a check cause incoming data to back up (consumer lag).

Solution: Use fast in-memory caches like Redis or RocksDB to validate records in under 1 millisecond.

2. Out-of-Order Data

Problem: Network delays cause events to arrive out of sequence, triggering false validation errors.

Solution: Use Event-Time and Watermarks (in Flink or Spark) to buffer and sort data based on when it actually happened.

3. Memory Overload (Deduplication)

Problem: Storing every unique ID in memory to check for duplicates eventually causes out-of-memory (OOM) crashes.

Solution: Apply a Time-To-Live (TTL) to auto-delete stored IDs after a set window (e.g., 2 hours).

4. Schema Drift (Format Changes)

Problem: Upstream teams change field names or data types without warning, crashing the stream.

Solution: Enforce strict data contracts using Avro/Protobuf and a Schema Registry to reject bad formats at the source.

5. Handling Broken Records

Problem: Crashing on bad data halts the app; silently dropping it causes permanent data loss.

Solution: Route bad events to a Dead Letter Queue (DLQ) for later fixing, while valid data continues downstream without interruption.

Summary

Production-Ready Completion Guide

This section follows the same learning progression used in the Managed Airflow documentation: introduction → architecture → core concepts → UI/setup → configuration → security → monitoring → troubleshooting → performance/cost → reliability → production design → end-to-end scenario → common mistakes → operational checklist.

A. Standard learning path

6. Sanity-check architecture

A real-time sanity-check layer should sit between ingestion and downstream consumption without becoming the processing bottleneck.

- Define synchronous vs asynchronous checks.
- Separate validation failures from infrastructure failures.
- Keep original events available for replay where policy permits.

7. Latency and throughput design

Validation needs an explicit latency budget.

- Use bounded-time lookups/caches.
- Measure p50/p95/p99 latency where useful.
- Avoid unbounded synchronous calls.

8. Event-time, watermarks and late data

Out-of-order events require a policy for event time, processing time and allowed lateness.

- Define lateness window.
- Use watermarks where supported.
- Document late-record behavior.

9. Deduplication and state management

Deduplication state must be bounded.

- Define key and TTL.
- Define behavior outside the window.
- Monitor state size and memory pressure.

10. Schema validation and contract management

Schema checks should be treated as data contracts.

- Define required fields/types.
- Version schemas where appropriate.
- Define compatibility rules.
- Quarantine schema-invalid records.

11. Bad-record handling and replay

Bad records should not silently disappear or unnecessarily stop the stream.

- Capture payload/failure reason where policy allows.
- Provide replay/reprocessing.
- Track DLQ volume and age.

12. Observability and alerts

Sanity checks should produce operational metrics, not only logs.

- Track validation failures, duplicates, schema failures, DLQ rate and consumer lag.
- Alert on sustained threshold breaches.
- Correlate failures with source changes.

13. Testing strategy

- Unit-test validation rules.
- Test late/out-of-order events.
- Test duplicate bursts and state expiry.
- Test schema changes.
- Test malformed records and DLQ.
- Test recovery/replay without duplicates.

14. Production checklist

- Define latency SLO/alert.
- Define lateness/watermark behavior.
- Define deduplication key/TTL.
- Define schema ownership/versioning.
- Define DLQ/replay process.
- Monitor state and lag.
- Document failure runbooks.

Official references

<https://docs.cloud.google.com/dataflow/docs/>

<https://docs.cloud.google.com/pubsub/docs/dead-letter-topics>

Final learning outcome

After completing this document, the learner should be able to explain real-time data sanity checks, create or configure its main resources, operate it from the Google Cloud console, troubleshoot common failures, and describe production controls.

Reference Tables

Problem	Root Cause	Solution
High Latency	Slow database lookups	In-memory cache (Redis)
Out-of-Order Data	Network delays	Event-time watermarks
Memory Crash	Storing too many IDs	Expire old keys via TTL
Schema Drift	Unexpected format changes	Schema Registry (Avro)
Bad Records	Unhandled bad data	Dead Letter Queue (DLQ)

Stage	Expected outcome
1. Introduction	Explain the service, problem and role.
2. Architecture	Explain components and flow.
3. Core concepts	Explain hierarchy, execution and terminology.
4. UI and setup	Create resources and recognize key settings.
5. Configuration	Configure runtime, identity, network and scaling.
6. Security	Apply IAM, encryption, secrets and least privilege.
7. Monitoring	Use logs, metrics and alerts.
8. Troubleshooting	Move from symptom to evidence to root cause.
9. Performance and cost	Relate design to throughput, latency and cost.
10. Reliability	Handle retries, duplicates and recovery.
11. Production design	Apply CI/CD, environment separation and ownership.
12. End-to-end scenario	Explain the service in a realistic pipeline.